

COMPLETE ROADMAP & COURSEWORK

Svelte

The compiler framework that ships no framework

Why Svelte exists, how its compiler works, reactivity, stores, transitions, SvelteKit, and the full ecosystem

7 Phases · Roadmap + Full Coursework with Code · Complete Reference

7 PHASES	40+ CONCEPTS	18 PROJECTS	~4mo TIMELINE	0kb FRAMEWORK RUNTIME
-------------	-----------------	----------------	------------------	-----------------------------

PHASE 0 Why Svelte Exists*The shift from runtime to compile-time — why this changes everything*

0.1 The Framework Runtime Problem

React, Vue, and Angular all ship a runtime to the browser. That runtime interprets your component code, manages a virtual DOM, and figures out what to update. This costs kilobytes, CPU cycles, and mental overhead.

Rich Harris (creator of Svelte, now at Vercel) asked: what if the framework was a compiler? What if we did all that work at build time, and shipped vanilla JavaScript that updates the DOM directly — no virtual DOM, no runtime, no overhead?

	React / Vue / Angular	Svelte
Approach	Runtime framework — shipped to browser	Compiler — converts components to vanilla JS at build time
Virtual DOM	Yes — diffing algorithm computes changes	No — direct DOM updates, no diffing needed
Bundle size	Framework runtime always included (React ~45kb gzip)	Near zero framework code — just your compiled app
Reactivity	State changes → React reconciler → DOM update	Compile-time reactive assignments → direct DOM mutations
Learning curve	Large API surface: hooks, lifecycle, context, memo...	Minimal API: just write variables and the compiler handles it
Performance	Fast (virtual DOM is optimised)	Faster (no virtual DOM overhead, surgical updates)

Svelte's Core Insight

Reactivity in Svelte is just assignment. When you write `count = count + 1`, the Svelte compiler transforms that into: update count, then surgically update every DOM node that depends on count. There is no `useState`, no `setCount`, no virtual DOM. The compiler is the framework.

0.2 The Roadmap at a Glance

Phase 1 — Svelte Fundamentals Single-file components, reactivity, props, events, bindings, lifecycle.	Phase 2 — Reactivity System Reactive declarations (\$:), reactive statements, reactive stores.
Phase 3 — Components In Depth Slots, named slots, component events, context API, special elements.	Phase 4 — Svelte Stores writable, readable, derived, custom stores. The store contract.
Phase 5 — Motion & Transitions Built-in transitions (fade, fly, slide), animations, spring, tweened.	Phase 6 — SvelteKit File-based routing, load functions, form actions, SSR, SSG.
Phase 7 — Advanced & Ecosystem TypeScript, testing, actions, use:directive, advanced patterns.	

PHASE 1 Svelte Fundamentals

Single-file components, reactivity, props, and events

1.1 The Svelte Component

Every Svelte component lives in a `.svelte` file. It combines script, styles, and template in a single file — but unlike JSX, the template is real HTML, not JavaScript.

```
<!-- Counter.svelte -->
<script>
  let count = 0;                                // reactive by default

  function increment() {
    count += 1;                                  // assignment triggers DOM update
  }
  function decrement() { count -= 1; }
  function reset()    { count = 0; }
</script>

<!-- Template is real HTML -->
<div class="counter">
  <p>Count: {count}</p>
  <button on:click={increment}>+</button>
  <button on:click={decrement}>-</button>
  <button on:click={reset}>Reset</button>
</div>

<!-- Scoped CSS — only applies to this component -->
<style>
  .counter { display: flex; gap: 8px; align-items: center; }
  p { font-size: 1.5rem; font-weight: bold; }
  button { padding: 8px 16px; border-radius: 4px; cursor: pointer; }
</style>
```

No Virtual DOM

In React, `count += 1` doesn't update the DOM — you need `setCount(c => c+1)`. In Svelte, `count += 1` is compiled into code that directly updates every DOM node that uses `count`. The compiler instruments your assignments.

1.2 Props — Receiving Data from Parents

```
<!-- Card.svelte -->
<script>
```

```
// Props are declared with export let
export let title;           // required
export let subtitle = '';   // optional with default
export let variant = 'default'; // optional
export let count = 0;       // typed default (number)
</script>

<div class="card card-{variant}">
  <h2>{title}</h2>
  {#if subtitle}
    <p>{subtitle}</p>
  {/if}
  <span>Count: {count}</span>
</div>

<!-- Usage in a parent -->
<script>
  import Card from './Card.svelte';
  let items = 42;
</script>

<Card title="Hello" count={items} />
<Card title="World" subtitle="optional" variant="featured" />

<!-- Spread props - pass an object as props -->
<script>
  const cardProps = { title: 'Spread', subtitle: 'Demo', variant: 'blue' };
</script>
<Card {...cardProps} />
```

1.3 Reactivity — Assignments and Declarations

```
<script>
  let firstName = 'John';
  let lastName = 'Doe';

  // Reactive declaration - $: prefix
  // Re-runs whenever any dependency changes
  $: fullName = `${firstName} ${lastName}`;
  $: greeting = `Hello, ${fullName}!`;

  // Reactive statement (no assignment - just a side effect)
  $: console.log('Full name changed:', fullName);

  // Reactive block - multiple statements
  $: {
    const trimmed = fullName.trim();
    if (trimmed.length < 2) {
      console.warn('Name too short');
    }
    document.title = trimmed;
  }

  // Reactivity with arrays and objects:
```

```
// Svelte only tracks TOP-LEVEL assignments
let numbers = [1, 2, 3];

function addNumber() {
  numbers.push(4);      // NOT reactive — mutates without reassigning
  numbers = numbers;    // trick: reassign to trigger reactivity
  // OR:
  numbers = [...numbers, 4]; // idiomatic — spread creates new array
}

let user = { name: 'Alice', age: 30 };
function birthday() {
  user.age += 1;        // reactive! Svelte tracks property access on objects
  // user.age += 1 compiles to: user.age += 1; user = user;
}
</script>

<input bind:value={firstName} />
<input bind:value={lastName} />
<p>{greeting}</p>
```

1.4 Template Directives

```
<!-- Conditionals: {#if}, {:else if}, {:else} -->
{#if user}
  <p>Welcome, {user.name}</p>
{:else if isLoading}
  <Spinner />
{:else}
  <button on:click={login}>Log in</button>
{/if}

<!-- Loops: {#each} with index and key -->
{#each todos as todo, index (todo.id)}
  <!-- (todo.id) is the key — essential for correct DOM updates when list changes -->
  <li class:done={todo.done}>
    <span>{index + 1}. {todo.text}</span>
    <button on:click={() => toggleTodo(todo.id)}>Toggle</button>
  </li>
{:else}
  <li>No todos yet!</li>
{/each}

<!-- Async blocks: {#await} -->
{#await fetchUser(userId)}
  <p>Loading...</p>
{:then user}
  <p>{user.name}</p>
{:catch error}
  <p>Error: {error.message}</p>
{/await}

<!-- Dynamic HTML: {@html} — use with caution (XSS risk) -->
```

```
{@html markdownContent}
```

1.5 Events & Two-Way Bindings

```
<script>
  let name = '';
  let agreed = false;
  let colour = '#ff3e00';
  let volume = 0.5;
  let size = 'medium';
  let selected = [];

  function handleInput(event) {
    console.log(event.target.value);
  }
</script>

<!-- bind:value - two-way binding for inputs -->
<input bind:value={name} placeholder="Type here" />
<p>Hello, {name}!</p>

<!-- bind:checked - for checkboxes -->
<input type="checkbox" bind:checked={agreed} />
<p>Agreed: {agreed}</p>

<!-- bind:value on select -->
<select bind:value={size}>
  <option value="small">Small</option>
  <option value="medium">Medium</option>
  <option value="large">Large</option>
</select>

<!-- Multiple select -->
<select multiple bind:value={selected}>
  <option>A</option><option>B</option><option>C</option>
</select>

<!-- Event modifiers -->
<form on:submit|preventDefault={handleSubmit}>
  <!-- |preventDefault, |stopPropagation, |once, |passive, |capture -->
  <button type="submit">Submit</button>
</form>

<!-- Inline handlers -->
<button on:click={() => count++}>Click</button>
<input on:input={handleInput} on:blur={() => console.log('blurred')} />
```

Phase 1 Projects



Reactive Todo App

Build a todo app in pure Svelte with no libraries. Uses reactive declarations for filtered lists, `{#each}` with keys, `{#if}` for conditionals, and `bind:value` for the input. Notice how much less code it takes vs React.

Concepts: `let` reactivity · `$:` declarations · `{#each}` · `{#if}` · `bind:value` · event handlers



Live Character Counter

A textarea with live stats: character count, word count, reading time. All computed with `$:` reactive declarations. No functions called manually — everything updates on assignment.

Concepts: reactive declarations · `$:` · two-way binding · computed values

PHASE 2 The Reactivity System

\$: declarations, reactive stores, and lifecycle

2.1 Lifecycle Functions

```
<script>
  import { onMount, onDestroy, beforeUpdate, afterUpdate, tick } from 'svelte';

  let data = null;
  let div;
  let scrollY;

  // onMount - runs after component is first rendered to DOM
  // (never runs during SSR - safe for browser APIs)
  onMount(async () => {
    data = await fetchData();
    return () => {
      // Optional: return a cleanup function (runs on destroy)
    };
  });

  // onDestroy - runs when component is removed from DOM
  const interval = setInterval(() => tick(), 1000);
  onDestroy(() => clearInterval(interval));

  // beforeUpdate - runs before DOM updates (state has changed, DOM hasn't yet)
  beforeUpdate(() => {
    // save scroll position before DOM changes
    scrollY = div?.scrollTop ?? 0;
  });

  // afterUpdate - runs after DOM has been updated
  afterUpdate(() => {
    // restore scroll or focus after re-render
    div.scrollTop = scrollY;
  });

  // tick() - returns a promise that resolves after next DOM update
  async function focusAfterUpdate() {
    someValue = newValue;
    await tick(); // wait for Svelte to update the DOM
    inputElement.focus(); // now safe to access updated DOM
  }
</script>

<div bind:this={div}>Content</div>
```

2.2 Component Events — Dispatching Upward

```
<!-- Child.svelte -->
<script>
  import { createEventDispatcher } from 'svelte';

  const dispatch = createEventDispatcher();

  export let item;

  function handleDelete() {
    // Dispatch a custom event with optional detail payload
    dispatch('delete', { id: item.id, name: item.name });
  }

  function handleSelect() {
    dispatch('select', item);
  }
</script>

<div>
  <span>{item.name}</span>
  <button on:click={handleDelete}>Delete</button>
  <button on:click={handleSelect}>Select</button>
</div>

<!-- Parent.svelte -->
<script>
  import Child from './Child.svelte';
  let items = [{ id: 1, name: 'Alpha' }, { id: 2, name: 'Beta' }];

  function handleDelete(event) {
    const { id } = event.detail;
    items = items.filter(i => i.id !== id);
  }
</script>

{#each items as item (item.id)}
  <Child {item} on:delete={handleDelete} on:select={e => console.log(e.detail)} />
{/each}

<!-- Event forwarding — pass DOM events from child to parent -->
<!-- In Child: <button on:click>Click me</button> (no handler = forwarded) -->
<!-- In Parent: <Child on:click={handleClick} /> -->
```

2.3 Slots — Component Composition

```
<!-- Panel.svelte — component with slots -->
<script>
  export let title;
  export let collapsible = false;
  let open = true;
</script>
```

```
<div class="panel">
  <header>
    <!-- Named slot for custom header content -->
    <slot name="header">
      <!-- Default content if no 'header' slot provided -->
      <h3>{title}</h3>
    </slot>
    {#if collapsible}
      <button on:click={() => open = !open}>{open ? '▲' : '▼'}</button>
    {/if}
  </header>
  {#if open}
    <div class="body">
      <!-- Default slot - receives children -->
      <slot />
    </div>
    <footer>
      <!-- Named footer slot with fallback -->
      <slot name="footer"><p>No footer</p></slot>
    </footer>
  {/if}
</div>

<!-- Parent usage -->
<Panel title="Default" collapsible>
  <p>This goes into the default slot.</p>
  <svelte:fragment slot="footer">
    <button>Save</button>
    <button>Cancel</button>
  </svelte:fragment>
</Panel>

<Panel>
  <!-- Override the header slot entirely -->
  <h1 slot="header">Custom Header!</h1>
  <p>Body content here.</p>
</Panel>

<!-- Slot props - pass data from child slot to parent -->
<!-- In Child: <slot item={currentItem} index={i} /> -->
<!-- In Parent: <Child let:item let:index> -->
<!--           <p>{index}: {item.name}</p> -->
<!--           </Child> -->
```

PHASE 3 Svelte Stores*Reactive state that lives outside components*

3.1 writable — Mutable Shared State

```
// stores.js — create stores in a separate file
import { writable } from 'svelte/store';

// A writable store is an object with subscribe, set, and update
export const count = writable(0);
export const userList = writable([]);
export const theme = writable('light');

// Component.svelte — using the store
<script>
  import { count, theme } from './stores.js';
  import { onDestroy } from 'svelte';

  // Manual subscription — must unsubscribe on destroy
  let countValue;
  const unsubscribe = count.subscribe(value => { countValue = value; });
  onDestroy(unsubscribe);

  // Auto-subscription with $ prefix — Svelte handles subscribe/unsubscribe
  // $count is reactive — no manual subscribe/unsubscribe needed
</script>

<!-- $prefix auto-subscribes to any store -->
<p>Count: {$count}</p>
<p>Theme: {$theme}</p>

<button on:click={() => count.update(n => n + 1)}>Increment</button>
<button on:click={() => count.set(0)}>Reset</button>
<button on:click={() => theme.set($theme === 'light' ? 'dark' : 'light')}>
  Toggle Theme
</button>

<!-- Two-way binding to a store -->
<input bind:value={$theme} />
<!-- bind:value={$store} directly updates the store on input -->
```

3.2 readable & derived Stores

```
import { readable, derived } from 'svelte/store';

// readable — value can only be set inside the store (consumers can't change it)
```

```
export const time = readable(new Date(), function start(set) {
  const interval = setInterval(() => set(new Date()), 1000);
  return function stop() { clearInterval(interval); };
  // start is called when first subscriber attaches
  // stop (returned function) is called when last subscriber detaches
});

// derived — computed from one or more other stores
import { count } from './stores.js';

export const doubled = derived(count, $count => $count * 2);
export const message = derived(count, $count => `You clicked ${$count} times`);

// Derived from multiple stores
import { firstName, lastName } from './userStores.js';
export const fullName = derived(
  [firstName, lastName],
  ([$first, $last]) => `${$first} ${$last}`
);

// Derived with async (provides a set function)
export const userDetails = derived(userId, ($userId, set) => {
  fetch(`/api/users/${$userId}`)
    .then(r => r.json())
    .then(set);
  return () => { /* cleanup */ };
}, null); // initial value while loading
```

3.3 Custom Stores

```
// Custom stores encapsulate state and logic together
// Must have a subscribe method — can have any other methods

import { writable } from 'svelte/store';

function createTodoStore() {
  const { subscribe, set, update } = writable([]);

  return {
    subscribe, // expose subscribe so $todo works in components

    add(text) {
      update(todos => [...todos, { id: Date.now(), text, done: false }]);
    },

    toggle(id) {
      update(todos => todos.map(t => t.id === id ? { ...t, done: !t.done } : t));
    },

    remove(id) {
      update(todos => todos.filter(t => t.id !== id));
    },
  };
}
```

```
    clear() { set([]); },
  };
}

export const todoStore = createTodoStore();

// Component.svelte
<script>
  import { todoStore } from './todoStore.js';
</script>

<input on:keydown={e => e.key === 'Enter' && todoStore.add(e.target.value)} />
{#each $todoStore as todo (todo.id)}
  <li>
    <input type="checkbox" checked={todo.done}
      on:change={() => todoStore.toggle(todo.id)} />
    <span class:done={todo.done}>{todo.text}</span>
    <button on:click={() => todoStore.remove(todo.id)}>x</button>
  </li>
{/each}
```

Phase 3 Projects



Shopping Cart with Stores

Cart state in a custom writable store. Products fetched with a derived store. Cart total derived from cart items. Persist to localStorage by subscribing to the store.

Concepts: writable · derived · custom store · \$store auto-subscribe · localStorage

PHASE 4 Transitions, Animations & Actions*Svelte's built-in motion system and DOM actions*

4.1 Built-in Transitions

Svelte ships first-class transition support. No animation library needed for most UI animations.

```

<script>
  import { fade, fly, slide, scale, blur, crossfade } from 'svelte/transition';
  import { flip } from 'svelte/animate';
  let visible = true;
  let items = ['Apple', 'Banana', 'Cherry'];
</script>

<!-- in: runs on mount, out: runs on unmount -->
<button on:click={() => visible = !visible}>Toggle</button>

{#if visible}
  <!-- Simple fade -->
  <div transition:fade={{ duration: 300 }}>Fades in and out</div>

  <!-- Fly in from below, fade out -->
  <div in:fly={{ y: 20, duration: 400 }} out:fade={{ duration: 200 }}>
    Flies in, fades out
  </div>

  <!-- Slide open/close -->
  <div transition:slide={{ duration: 300 }}>Slides in</div>
{/if}

<!-- animate:flip for list reordering -->
{#each items as item (item)}
  <div animate:flip={{ duration: 300 }}>{item}</div>
{/each}

<!-- Custom transition function -->
<script>
  function typewriter(node, { speed = 40 }) {
    const text = node.textContent;
    const duration = text.length * speed;
    return {
      duration,
      tick: t => { node.textContent = text.slice(0, Math.floor(t * text.length)); }
    };
  }
</script>
<p in:typewriter={{ speed: 30 }}>This text types itself in!</p>

```

4.2 Spring & Tweened Stores

```
<script>
  import { spring, tweened } from 'svelte/motion';
  import { cubicOut, elasticOut } from 'svelte/easing';

  // tweened — animates to a target value over a fixed duration
  const progress = tweened(0, { duration: 800, easing: cubicOut });

  function setProgress(value) {
    progress.set(value); // value animates smoothly to the target
  }

  // spring — physics-based animation (stiffness, damping)
  const coords = spring({ x: 0, y: 0 }, { stiffness: 0.1, damping: 0.8 });

  function handleMouseMove(e) {
    coords.set({ x: e.clientX, y: e.clientY }); // follows mouse with spring physics
  }
</script>

<div on:mousemove={handleMouseMove}>
  <!-- $coords auto-subscribes — updates on every frame during animation -->
  <div style="transform: translate({$coords.x}px, {$coords.y}px)">□</div>
</div>

<progress value={$progress} max={100} />
<button on:click={() => setProgress(75)}>Set to 75%</button>
```

4.3 Actions — use: Directive

Actions are functions that run when an element is added to the DOM. They can set up event listeners, integrate third-party libraries, and return cleanup functions.

```
// actions.js
// An action receives the DOM node and optional parameters

// Trap focus within a modal
export function focusTrap(node) {
  const focusable = node.querySelectorAll(
    'button, input, select, textarea, [tabindex]:not([tabindex="-1"])'
  );
  const first = focusable[0];
  const last = focusable[focusable.length - 1];

  function handleKeydown(e) {
    if (e.key !== 'Tab') return;
    if (e.shiftKey && document.activeElement === first) {
      e.preventDefault(); last.focus();
    }
  }
}
```



```

    } else if (!e.shiftKey && document.activeElement === last) {
      e.preventDefault(); first.focus();
    }
  }

  node.addEventListener('keydown', handleKeydown);
  first?.focus();
  return { destroy: () => node.removeEventListener('keydown', handleKeydown) };
}

// Action with parameters
export function tooltip(node, text) {
  let tip;
  function show() {
    tip = document.createElement('div');
    tip.textContent = text;
    tip.className = 'tooltip';
    document.body.appendChild(tip);
  }
  function hide() { tip?.remove(); }
  node.addEventListener('mouseenter', show);
  node.addEventListener('mouseleave', hide);
  return {
    update(newText) { text = newText; }, // called when parameter changes
    destroy() { hide(); node.removeEventListener('mouseenter', show); }
  };
}

<!-- Usage in component -->
<script>
  import { focusTrap, tooltip } from './actions.js';
</script>
<dialog use:focusTrap>...</dialog>
<button use:tooltip={'Click to save'}>Save</button>

```

Phase 4 Projects



Animated Todo List

Extend the todo app: items fly in when added (fly transition), fade out when deleted, and smoothly reorder with `animate:flip`. The whole thing should feel alive.

Concepts: `transition:fly` · `transition:fade` · `animate:flip` · `spring` · `stores`

PHASE 5 SvelteKit*Full-stack Svelte: routing, SSR, forms, and deployment*

5.1 File-Based Routing

SvelteKit uses the filesystem as the router. Every `+page.svelte` file in `src/routes/` becomes a URL. Every `+layout.svelte` wraps child routes.

```
src/routes/
+page.svelte           → /
+layout.svelte         → wraps all routes (nav, footer)
about/
+page.svelte           → /about
blog/
+page.svelte           → /blog
[slug]/
+page.svelte           → /blog/my-post
+page.server.ts        → server-side load for that route
(auth)/
login/+page.svelte     → /login
register/+page.svelte  → /register
api/
users/+server.ts       → /api/users (API endpoint)

// File roles:
// +page.svelte         - the page component
// +page.ts             - universal load function (runs client + server)
// +page.server.ts      - server-only load (can access DB, env vars)
// +layout.svelte       - layout wrapping all child routes
// +layout.server.ts    - server load for the layout
// +server.ts           - API route (GET, POST, PUT, DELETE handlers)
// +error.svelte        - custom error page
```

5.2 Load Functions

```
// +page.server.ts - runs ONLY on the server
import type { PageServerLoad } from './$types';
import { db } from '$lib/server/db';

export const load: PageServerLoad = async ({ params, cookies, fetch }) => {
  const session = cookies.get('session');
  if (!session) redirect(302, '/login');

  const post = await db.post.findUnique({ where: { slug: params.slug } });
  if (!post) throw error(404, 'Post not found');
```

```
    return { post }; // passed as 'data' prop to +page.svelte
  };

// +page.svelte - receives the data
<script>
  export let data; // typed as PageData if using $types
</script>
<h1>{data.post.title}</h1>
<article>{<html data.post.content></article>

// +page.ts - universal load (runs on server for SSR, then client for navigation)
import type { PageLoad } from './$types';

export const load: PageLoad = async ({ fetch, params }) => {
  // fetch is SvelteKit's enhanced fetch - works server and client side
  const res = await fetch(`/api/posts/${params.slug}`);
  return { post: await res.json() };
};
```

5.3 Form Actions — Server-Side Mutations

```
// +page.server.ts - define form actions
import type { Actions } from './$types';
import { fail } from '@sveltejs/kit';

export const actions: Actions = {
  // Default action - handles <form method="POST">
  default: async ({ request, cookies }) => {
    const data = await request.formData();
    const email = data.get('email')?.toString();
    const password = data.get('password')?.toString();

    if (!email || !password) {
      return fail(400, { error: 'Email and password are required', email });
    }

    const user = await authenticate(email, password);
    if (!user) return fail(401, { error: 'Invalid credentials', email });

    cookies.set('session', user.sessionToken, { path: '/', httpOnly: true });
    redirect(303, '/dashboard');
  },
}

// +page.svelte - the form
<script>
  export let form; // contains the return value from a failed action
</script>

<form method="POST">
  {#if form?.error}
    <p class="error">{form.error}</p>
  {/if}
```

```
<input name="email" value={form?.email ?? ''} type="email" required />
<input name="password" type="password" required />
<button>Log In</button>
</form>

// Progressive enhancement with use:enhance
<script>
  import { enhance } from '$app/forms';
</script>
<form method="POST" use:enhance>
  <!-- Works without JS (full page reload), enhanced with JS (no reload) -->
</form>
```

5.4 Rendering Modes

```
// +page.server.ts or +page.ts

// Default: SSR (Server-Side Rendering) — HTML rendered on server per request

// SSG (Static Site Generation) — pre-render at build time
export const prerender = true;

// Disable SSR — render only in browser (CSR)
export const ssr = false;

// ISR-style: prerender + revalidation (edge functions)
// Configured in adapter settings

// +layout.ts — apply to all routes in a directory
export const prerender = true;
export const ssr = true;

// SvelteKit adapters determine output format:
// adapter-auto      → detects Vercel, Netlify, Cloudflare automatically
// adapter-node      → standalone Node.js server
// adapter-static     → fully static site (requires prerender=true everywhere)
// adapter-vercel     → Vercel Edge Functions and serverless
// adapter-cloudflare → Cloudflare Workers
```

Phase 5 Projects



SvelteKit Blog

A full blog with SvelteKit: SSR for post pages, static prerendering for the listing page, Markdown rendering, dynamic [slug] routes, and an API endpoint that returns posts as JSON.

Concepts: SvelteKit routing · SSR · prerender · load functions · \$types



Auth + CRUD App

A full CRUD app with SvelteKit form actions: register, login with cookie sessions, create/edit/delete records. Progressive enhancement with use:enhance. No client-side JS required for core features.

Concepts: form actions · cookies · fail/redirect · use:enhance · progressive enhancement

PHASE 6 TypeScript & Testing in Svelte

Type safety and confidence in Svelte components

6.1 TypeScript in Svelte Components

```
<!-- TypedCounter.svelte -->
<script lang="ts">
  interface Item {
    id: number;
    label: string;
    done: boolean;
  }

  // Typed props
  export let items: Item[] = [];
  export let maxItems: number = 100;
  export let onComplete: (id: number) => void = () => {};

  // Typed reactive declarations
  $: completed = items.filter((i: Item) => i.done);
  $: remaining = items.filter((i: Item) => !i.done);

  // Typed event handler
  function handleKeyDown(e: KeyboardEvent) {
    if (e.key === 'Enter') {
      const input = e.target as HTMLInputElement;
      addItem(input.value);
      input.value = '';
    }
  }

  function addItem(label: string): void {
    if (items.length >= maxItems) return;
    items = [...items, { id: Date.now(), label, done: false }];
  }
</script>

<input on:keydown={handleKeyDown} placeholder="Add item..." />
<p>{remaining.length} remaining, {completed.length} done</p>
{#each items as item (item.id)}
  <div>{item.label}</div>
{/each}
```

6.2 Testing with Vitest & Svelte Testing Library

```
// npm install -D vitest @testing-library/svelte jsdom
// vitest.config.ts:
```

```
// import { defineConfig } from 'vitest/config';
// import { svelte } from '@sveltejs/vite-plugin-svelte';
// export default defineConfig({ plugins: [svelte({ hot: !process.env.VITEST })],
//   test: { environment: 'jsdom' } });

// Counter.test.ts
import { render, screen, fireEvent } from '@testing-library/svelte';
import { describe, it, expect } from 'vitest';
import Counter from './Counter.svelte';

describe('Counter', () => {
  it('renders initial count of 0', () => {
    render(Counter);
    expect(screen.getByText('Count: 0')).toBeTruthy();
  });

  it('increments when + button clicked', async () => {
    render(Counter);
    const btn = screen.getByRole('button', { name: '+' });
    await fireEvent.click(btn);
    expect(screen.getByText('Count: 1')).toBeTruthy();
  });

  it('accepts an initial count prop', () => {
    render(Counter, { props: { initialCount: 5 } });
    expect(screen.getByText('Count: 5')).toBeTruthy();
  });
});

// Testing stores
import { get } from 'svelte/store';
import { todoStore } from './todoStore.js';

describe('todoStore', () => {
  it('adds a todo', () => {
    todoStore.add('Write tests');
    expect(get(todoStore)).toHaveLength(1);
    expect(get(todoStore)[0].text).toBe('Write tests');
  });
});
```

Phase 6 Projects



TypeScript Svelte Component Library

Rebuild the todo app components fully typed with TypeScript. Add type-safe stores (`writable<TodoItem[]>`), typed event dispatchers, and typed slot props. Write tests for every component and store.

Concepts: lang=ts · typed props · typed stores · createEventDispatcher · Vitest · Svelte Testing Library

PHASE 7 Advanced Patterns & Ecosystem*Context API, special elements, and the full Svelte ecosystem*

7.1 Context API — Share Without Stores

Svelte's context API shares data within a component subtree without passing props through every level. Unlike stores, context is not globally reactive — it is scoped to the component tree.

```

<!-- DataProvider.svelte -->
<script>
  import { setContext } from 'svelte';
  import { writable } from 'svelte/store';

  // Create a store and put it in context
  const user = writable(null);
  setContext('user', user);

  export async function loadUser(id) {
    const data = await fetchUser(id);
    user.set(data);
  }
</script>
<slot />

<!-- DeepChild.svelte - anywhere in the subtree -->
<script>
  import { getContext } from 'svelte';
  const user = getContext('user');
</script>
<p>Hello, {$user?.name ?? 'Guest'}</p>

// Context vs Stores:
// Stores: global singleton - same value everywhere in the app
// Context: scoped to a subtree - each instance can have different data
// Use context for: table/row relationships, map/marker, form/field

```

7.2 Special Svelte Elements

```

<!-- svelte:self - recursive components (trees, nested lists) -->
<script>
  export let tree;
</script>
<li>{tree.name}
  {#if tree.children?.length}
    <ul>{#each tree.children as child}<svelte:self tree={child} />{/each}</ul>
  {/if}

```



```

</li>

<!-- svelte:component — dynamic component rendering -->
<script>
  import Foo from './Foo.svelte';
  import Bar from './Bar.svelte';
  let CurrentComponent = Foo;
</script>
<svelte:component this={CurrentComponent} someProp={value} />

<!-- svelte:window — attach event listeners to window -->
<svelte:window on:keydown={handleKey} bind:scrollTop={y} bind:innerWidth={w} />

<!-- svelte:head — inject into <head> (for SEO, meta tags) -->
<svelte:head>
  <title>{post.title} | My Blog</title>
  <meta name="description" content={post.excerpt} />
</svelte:head>

<!-- svelte:body — attach listeners to document.body -->
<svelte:body on:mouseenter={() => active = true} on:mouseleave={() => active = false} />

```

7.3 Svelte vs React — When to Choose Each

Dimension	Svelte	React
Bundle size	Tiny — no framework runtime shipped	Larger — React runtime always included
Performance	Surgical DOM updates, no VDOM overhead	Fast, but VDOM reconciliation has overhead
Learning curve	Lower — closer to HTML/CSS/JS you already know	Higher — JSX, hooks, immutability rules
Reactivity model	Compile-time: just assign variables	Runtime: explicit useState and setters
TypeScript	Supported with lang="ts"	Excellent — most React projects use TS
Ecosystem	Growing but smaller	Massive — years of libraries and patterns
SSR / Full-stack	SvelteKit — first-class, well designed	Next.js — industry standard, very mature
Jobs market	Growing rapidly, still niche	Dominant — most frontend jobs
Best for	Small-medium apps, performance-critical UIs	Large teams, enterprise, complex SPAs

Capstone Projects



SvelteKit Movie App

A full SvelteKit app: movie listing (SSR), movie detail page (SSR + dynamic routes), search (client-side with stores), watchlist (writable store + localStorage), and a REST API endpoint.

Concepts: SvelteKit · SSR · stores · load functions · API routes · TypeScript



Real-Time Dashboard

A live data dashboard using WebSockets: connect on mount (onMount/onDestroy), data in writable stores, animated charts with tweened stores, multiple panels using context to share the WebSocket.

Concepts: WebSockets · onMount · writable stores · tweened · context · transitions

You have reached the end of the Svelte coursework.

Svelte makes you feel like a superhero. Now go build something and show it off.